

Mini Hospital Emergency Management System

Demo Video Script

1. Introduction

Hello everyone.

My name is Jesla.

Today, I am going to demonstrate my individual mid assignment project.

The project title is **Mini Hospital Emergency Management System Using Data Structures**.

I developed this project using Java.

The main purpose of this system is to manage hospital patient records, emergency patients, treatment history, and patient visit history.

In this project, I used four main data structures:

1. Binary Search Tree
2. Queue
3. Stack
4. Singly Linked List

Now I will demonstrate each part of the system.

2. GitHub Repository

First, I will show my GitHub repository.

This is my GitHub repository called **MiniHospitalEmergencySystem**.

The repository is public.

Here, I have uploaded my Java source files, README file, and .gitignore file.

I also made multiple commits during the development of the project.

The README file explains the project description, data structures used, features, technologies, and how to run the program.

3. Patient Records – Binary Search Tree

Now I will explain the patient records.

For patient records, I used a **Binary Search Tree**, or BST.

Each patient has a patient ID, name, age, contact number, and medical condition.

I inserted five patients into the BST.

The patient IDs are 101, 102, 103, 104, and 105.

The system displays the patients using **in-order traversal**.

Because of the BST, the patient IDs are displayed in ascending order.

I also implemented a search operation.

Here, I search for patient ID 103.

The system successfully finds patient 103, whose name is John.

I also implemented the delete operation.

Here, I delete patient 105.

After deletion, patient 105 is no longer displayed in the patient records.

This demonstrates the main BST operations: insert, search, delete, and in-order display.

4. Emergency Patient Queue

Next, I will demonstrate the emergency patient queue.

For emergency patients, I used a **Queue**.

A queue follows the **FIFO principle**, which means First In, First Out.

I added three patients to the emergency queue.

They are patient 103, patient 101, and patient 104.

The queue displays them in the same order.

When I treat the next patient, patient 103 is treated first because patient 103 was added first.

After treatment, patient 103 is removed from the queue.

Then patient 101 is treated next.

After that, patient 104 remains in the queue.

This demonstrates the FIFO behavior of the Queue.

5. Treatment History – Stack

Next, I will explain the treatment history.

For treatment history, I used a **Stack**.

A stack follows the **LIFO principle**, which means Last In, First Out.

After treating patients 103 and 101, I added them to the treatment history stack.

The most recently treated patient is placed on the top of the stack.

Therefore, the treatment history demonstrates the LIFO behavior of a Stack.

This is useful when we want to access the most recent treatment first.

6. Patient Visit History – Singly Linked List

Next, I will demonstrate the patient visit history.

For this part, I used a **Singly Linked List**.

Each visit contains:

- Visit ID
- Visit date
- Doctor name
- Diagnosis
- Treatment

I added three visits to the linked list.

The system displays all the previous visits.

I also implemented a search operation.

Here, I search for visit ID 2.

The system successfully finds visit 2 and displays its details.

I also implemented the remove operation.

Here, I remove visit ID 1.

After removing visit 1, the system displays the remaining visits, which are visit 2 and visit 3.

This demonstrates add, search, remove, and display operations using a Singly Linked List.

7. Final Output

Now I will show the final output of the complete system.

The program successfully demonstrates:

- Patient registration
- Patient search
- Patient deletion
- Emergency queue management
- Patient treatment
- Treatment history
- Patient visit history
- Visit search
- Visit removal

There are no errors when running the program.

8. Conclusion

To conclude, this project helped me understand how different data structures can be used in a real-world application.

I used a Binary Search Tree for patient records, a Queue for emergency patients, a Stack for treatment history, and a Singly Linked List for patient visit history.

Through this project, I improved my understanding of Java, data structures, and GitHub version control.

Thank you for watching my demonstration.